

Generating Tool-Capable, Behavior-Stratified Model Pools for Adjudication

Jamie Stephens

August 30, 2026

Abstract

This report specifies an end-to-end pipeline for constructing adjudication model pools whose members can use the runtime’s tools, follow its response protocol, answer baseline reasoning questions, and differ in observed behavior. The pipeline treats a model-provider endpoint as the evaluation unit, pins provider routing, verifies direct function calling and Pi/MCP use, scores repeated adjudication-oriented responses, embeds multiple responses to 14 behavior prompts, reduces and clusters each prompt’s embedding vectors, and samples uniformly over cluster-assignment tuples while enforcing endpoint equivalence and one model per record. A recorded 2026 run evaluated 419 screened configurations, retained 283 with no provider errors and a deliberation score greater than 0.70, excluded 18 after gene-stage request failures, and produced 100 records from 265 eligible configurations. The pool contained 100 model identifiers, 30 providers, and 69 observed cluster tuples. Catalog drift, prompt selection, stochastic responses, and embedding-based clustering limit claims beyond the recorded run.

1 Introduction

A model identifier does not specify one inference system. A routing service may offer the same identifier through several providers, quantizations, regions, service classes, and parameter sets. These endpoints can differ in availability, tool-call behavior, latency, context limits, and response content. Selecting only by model identifier therefore leaves the executed system unspecified.

Adjudication adds two requirements. A selected configuration must use the same submission tools and agent harness that it will encounter during a case, and a pool should avoid concentrating all seats in configurations with the same observed response pattern. Catalog metadata helps identify candidates, but it does not establish that a model will complete a tool call through the deployed request path. A factual or reasoning score records performance on a fixed set, but it does not describe variation among configurations that pass the score threshold.

The pipeline described here constructs an auditable pool by evaluating pinned model-provider endpoints through runtime tool paths, filtering repeated adjudication responses, and sampling over vectors of per-prompt behavior clusters. It preserves the catalog row, request route, raw response records, scores, embeddings, cluster assignments, aggregation decisions, and sampling decisions. The implementation resides in the AgentCourt `adj` repository at revision `36316f3` [1].

The recorded run reduces a large endpoint catalog to a tool-capable, scored, and behavior-stratified pool while retaining a record of each exclusion. The run does not estimate the quality of later verdicts, which requires procedure-level evaluation with cases and outcomes.

2 Selection Unit and Output

2.1 Endpoint configurations

The pipeline evaluates an endpoint configuration rather than a model name. A configuration contains the OpenRouter model identifier, canonical model slug, provider name and route tag, quantization, input and output modalities, context and completion limits, supported request parameters, pricing, recent service measurements, and source-catalog identity. The inventory joins the OpenRouter model catalog with the endpoint list for each selected model [2, 3].

Let

$$c = (m, p, q, a, s),$$

where m is the OpenRouter model identifier, p is the provider route, q is the advertised quantization, a contains the model and endpoint attributes, and s identifies the catalog snapshot and row. The implementation assigns an endpoint-variant identifier of the form

$$\text{openrouter} : m@p\#q.$$

The request sets the provider order to the selected route, disables fallbacks, and requires support for the sent parameters. OpenRouter documents these routing controls through the request’s provider object [4].

Some catalog rows share every request-selectable route field. The API can select the model, provider route, and known quantization, but it cannot select one row within such a group. The inventory marks every member of the group as an ambiguous exact route, and the screening and evaluation coordinators reject it before inference. This rule prevents a recorded endpoint identity from claiming precision that the request API cannot enforce.

2.2 Pool records

Each output row contains the selected endpoint configuration, a persona reference, one aggregate cluster label for each behavior prompt, the sample-level assignments behind each aggregate label, and any equivalent endpoints grouped with the selected representative. Runtime code uses the endpoint fields to reconstruct the pinned request. It resolves the persona path when it creates a council member or juror.

The pool is a JSON Lines file because each row is a complete selection record. The default build requests 100 rows, samples without replacement, and permits one row per OpenRouter model identifier. These constraints reduce exact duplication while preserving provider and model-family information for later analysis.

3 Pipeline

Table 1 states each stage’s input and output. The runner writes every generated file beneath one run directory and writes a summary after each completed stage.

3.1 Catalog inventory

The inventory fetches the model catalog and then fetches every endpoint list for the selected model identifiers. A full-catalog run uses every catalog model. A smaller run may accept explicit model identifiers or sample root models with a recorded seed. OpenRouter’s model and endpoint APIs

Table 1: Pool-generation stages.

Stage	Input	Principal output
Inventory	Model catalog and per-model endpoint catalog	Normalized endpoint configurations and raw catalog responses
Tool screen	Endpoint configurations	Accepted and rejected configurations, direct calls, Pi transcripts, MCP calls, usage, and cost
Evaluation	Accepted configurations, question set, prompt, and trial count	Raw responses, deterministic scores, and per-configuration summaries
Score filter	Evaluation summaries and endpoint configurations	Configurations meeting the error and score criteria
Gene collection	Filtered configurations, persona, behavior prompts, and sample count	Completions, embeddings, request metadata, failures, and cost
Cross-gene filter	Every gene record	One configuration set complete across every prompt and sample
PCA and clustering	Per-gene embeddings	Reduced coordinates, cluster fits, and sample-level labels
Aggregation	Sample-level labels	One cluster-label vector for each configuration and persona
Sampling	Aggregate vectors and endpoint metadata	Pool records, equivalence records, and selection diagnostics

expose the identifiers, modalities, parameter support, context limits, pricing, and provider-specific endpoint records used here [2, 3].

Normalization assigns a monotonically increasing combined index to each endpoint row. It retains the raw responses beside the normalized JSONL and CSV forms, allowing later checks against the source data. Endpoint-fetch failure stops inventory because a partial catalog would make absence indistinguishable from a request failure. Rows with no available endpoint remain visible through inventory status fields and fail later eligibility checks.

The implementation partitions subsequent work by model identifier. All endpoints for one model remain in one partition, and a greedy size-balancing step distributes those groups among the requested number of operating-system processes. The default `make pool` command uses eight screen processes and eight evaluation processes. Each screen process handles its configurations sequentially, while the parent monitors process exits and stops sibling processes after an infrastructure failure.

3.2 Runtime tool screen

Metadata screening requires a model identifier, an unambiguous provider route, text input, text output, and advertised `tools` support. A configuration that fails one of these checks receives a rejection record without an inference request. Advertising tool support is an initial condition because provider-specific implementations can still reject a tool schema or return text in place of a call.

Each remaining configuration must pass two live tests. The direct test sends the same strict `submit_council_vote` function schema used by the Quick procedure and requires exactly one call with a permitted vote and a nonempty rationale. OpenRouter describes this client-executed tool-call pattern as a request containing a function schema followed by a model response containing the requested function and arguments [5]. The screen uses the selected provider route, disables fallback

routing, and applies bounded request attempts.

The second test starts Pi in the same container image and with the same OpenRouter model definition used by the ARB-family runtimes. The screen starts a local Model Context Protocol (MCP) server, exposes `wait_for_opportunity` and `submit_council_vote`, and gives Pi a signed capability for council seat C1. Pi must call the wait tool and then submit the requested vote through its MCP proxy. MCP defines model-discoverable tools with JSON Schema inputs and `tools/list` and `tools/call` messages [6]; Pi supplies the agent process and its tool loop [7].

The screen accepts a configuration only when both paths pass. Candidate failures, including malformed tool arguments and provider-attributed request errors, become rejection records. Credential failures, container failures, transcript failures, and unattributed authentication failures stop the affected partition because they prevent a valid judgment about the configuration. Each accepted or rejected row retains the direct response, Pi transcript, MCP log, usage, observed cost, elapsed time, and classified error.

3.3 Question-set evaluation

The evaluation runner sends a fixed question set through each accepted configuration for T trials. The default build uses 20 questions and $T = 3$. The set has four questions in each of five categories: basic human knowledge, basic science and quantitative reasoning, basic reasoning, instruction following, and tool-based evidence use. Ordinary responses contain an answer, confidence, rationale, and an empty evidence list. Record-based responses contain a vote, confidence, rationale, and cited evidence identifiers.

The response schema permits only the required fields and constrains confidence to $[0, 1]$. Record-based tasks expose evidence-listing and evidence-reading tools. A completed record-based response passes the tool check only when its trace contains both required tools and no disallowed tool. The scorer also checks answer correctness, the configured deliberation answer, item-specific instructions, rationale length, evidence references, malformed JSON, schema violations, invalid votes, timeouts, context errors, provider errors, latency, and cost.

Let $D_t(c)$ be the completed deliberation rows for configuration c in trial t , excluding rows with response-metadata errors. If $y_i = 1$ when row i matches the configured deliberation answer and 0 otherwise, the trial score is

$$s_t(c) = \frac{1}{|D_t(c)|} \sum_{i \in D_t(c)} y_i.$$

The deliberation score is the mean over trials that contain at least one completed deliberation row:

$$S(c) = \frac{1}{|T_c|} \sum_{t \in T_c} s_t(c).$$

This trial-first mean gives each represented trial equal weight. The deliberation score uses the 12 questions in the first three categories. The scorer records instruction following and tool-based evidence use as separate dimensions. The score output also records the aggregate row fraction, trial standard deviation, range, and item-level variation across trials.

The provider-error count combines provider, rate-limit, credential, and runner errors. The default filter requires a successful evaluator exit, zero provider errors, and $S(c) \geq 0.90$. The recorded run analyzed below used the explicit condition $S(c) > 0.70$. Schema, tool, timeout, latency, and cost measurements remain in the score record but do not independently exclude a configuration under this filter.

3.4 Behavior prompts and embeddings

Configurations that pass the score filter receive behavior-eliciting prompts called genes. A gene is an ordinary text prompt intended to reveal differences in subject, framing, self-description, values, or style while the persona remains fixed. The default source contains 14 prompts concerning social issues, religion, macroeconomics, nature, justice, training provenance, values, guidelines, surveillance, leadership, metaphor, and climate. The prompts do not contain case facts.

For each eligible configuration c , gene g , and sample index $r \in \{1, \dots, R\}$, the runner sends the same persona and sampling parameters through the pinned endpoint. The default build uses $R = 5$, temperature 0.7, top- p 1.0, and at most 512 output tokens. It then maps each successful response text to an embedding

$$x_{cgr} \in \mathbf{R}^{1536}$$

with `text-embedding-3-small`. OpenAI describes embeddings as vectors whose distances measure text relatedness and lists clustering and diversity measurement among their uses [8].

Each gene command records every expected sample, including completion and embedding failures, and uses bounded completion retries. The parent may run a specified number of gene processes at once; the default build uses two. After every gene finishes, the cross-gene filter excludes a configuration from all downstream stages if any selected gene has a failed completion, failed embedding, missing record, missing sample index, or missing embedding. This shared eligible set ensures that every configuration contributes the same number of samples to every gene.

3.5 Per-gene PCA and clustering

The pipeline fits principal component analysis (PCA) independently for each gene. It centers the $N \times 1536$ embedding matrix, computes its singular value decomposition, retains the first d right singular vectors, and projects each embedding into $\mathbf{R}^d$. The default build uses $d = 8$. PCA orders the axes by explained variance and represents each centered record through its coordinates along those axes [9].

For each gene, the pipeline fits K-means for every integer k from 2 through 20. Each fit uses ten initializations and random state 0. K-means assigns each record to the closest fitted center under Euclidean distance [10]. The pipeline selects the k with the greatest mean silhouette coefficient, which compares within-cluster cohesion with separation from the nearest other cluster [11]. Iteration proceeds in increasing k , so an exact score tie retains the smaller k .

This stage clusters response samples rather than configurations. A configuration therefore receives five sample-level labels for each gene. Cluster labels have meaning only within one gene and one fitted run; label 0 for one gene has no relation to label 0 for another gene.

3.6 Configuration-level cluster vectors

Aggregation converts the R sample labels for each configuration-gene pair into one label. A unique modal label wins. If several labels tie for the largest count, the algorithm considers samples assigned to the tied clusters and selects the label of the sample closest to its fitted cluster center. Sample index and cluster number provide deterministic final tie breaks.

For G genes, each configuration and persona receives the vector

$$z(c) = (\ell_{c1}, \ell_{c2}, \dots, \ell_{cG}),$$

where ℓ_{cg} is the aggregate label for gene g . The record preserves the five sample labels, their counts, the aggregation method, and any tie-breaking sample. The vector serves as the stratum used by the pool sampler.

3.7 Endpoint equivalence and tuple sampling

Before sampling, the implementation groups endpoint rows by a model-equivalence key containing the OpenRouter model identifier, endpoint model identifier, canonical slug, Hugging Face identifier when present, quantization, and input and output modalities. Provider identity is excluded from this key. The sampler selects one representative per group by an ordered comparison of recorded errors, deliberation score, schema and timeout counts, context capacity, recent uptime, recent latency, price, and stable endpoint identifiers. The output retains every member of the equivalence group, so a runtime or analyst can inspect the alternatives.

The sampler groups representatives by their complete cluster vector $z(c)$. At each step, it selects one currently available vector uniformly and then one currently available row within that vector uniformly. Without-replacement sampling removes the chosen row. The one-per-model constraint removes every remaining row with the chosen OpenRouter model identifier. A vector leaves the available set when these removals leave it with no row.

This procedure gives a rare observed cluster vector the same first-step selection probability as a common vector. It therefore targets representation across observed behavior strata instead of estimating their catalog prevalence. A fixed random seed makes the post-clustering sampling repeatable for the same input rows and order.

4 Recorded 100-Entry Pool

4.1 Run record

Inventory for the completed pool started on August 25, 2026. The OpenRouter catalog snapshot contained 419 model identifiers and 1,553 endpoint rows. The endpoint status field was 0 for 1,461 rows, -2 for 61 rows, and -5 for 31 rows. Six rows belonged to three ambiguous exact-route groups. A metadata filter used during that run retained 725 rows based on exact routing, text input and output, at least 64,000 context tokens, at least 4,096 completion tokens, and the request parameters then required by the evaluator.

Screening and evaluation ran in partitions and produced a consolidated 419-configuration source set. Every configuration in that set had passed both tool screens. The pool command then used its supported start-at-filter path, so the final run directory begins with the consolidated configurations and evaluation summaries. The source summary records 419 evaluation rows and \$36.616344 in observed evaluation cost. The retained screen output spans initial partitions and focused reruns and therefore does not support one screening cost or duration.

Table 2 gives the reductions after consolidation. The score filter retained 283 configurations with zero provider errors and $S(c) > 0.70$. Gene collection requested $283 \times 14 \times 5 = 19,810$ records. It produced 19,494 successful completions and embeddings, 316 completion-error records, and no embedding errors.

Eighteen configurations had at least one gene-stage error and were removed from every gene. Five had only rate-limit errors, twelve had only provider errors, and one had both. The resulting set contained 265 configurations and 1,325 records per gene. The observed OpenRouter completion cost for the gene stage was \$25.780985. Three successful completions lacked a cost observation, and the run record contains no embedding-cost total.

4.2 Clustering and sampling results

The eight PCA dimensions explained between 0.499 and 0.746 of embedding variance across the 14 separately fitted genes. Silhouette selection chose $k = 2$ for twelve genes, $k = 19$ for the prompt

Table 2: Recorded run counts after consolidation.

Stage	Input	Output	Exclusion or transformation
Evaluation filter	419	283	Zero provider errors and $S(c) > 0.70$
Gene collection	283	19,810 records	14 genes and five samples
Cross-gene filter	283	265	Complete success across every gene and sample
PCA and clustering	18,550 records	18,550 labels	Eight dimensions and per-gene k selection
Aggregation	18,550 labels	265 vectors	One label per configuration and gene
Equivalence grouping	265	148 representatives	Model-configuration equivalence key
Pool sampling	148	100	Seed 0, no replacement, one row per model

asking which organization trained the model, and $k = 10$ for the prompt asking for a European social issue. The run does not assign semantic names to any cluster.

Aggregation made $265 \times 14 = 3,710$ configuration-gene decisions. It found unanimous sample labels in 2,963 decisions, a unique nonunanimous majority in 697, and a tied mode requiring the center-distance rule in 50. Endpoint equivalence reduced 265 rows to 148 representatives with 83 distinct cluster vectors in the sampling frame.

The sampler emitted 100 rows containing 100 model identifiers, 100 endpoint-variant identifiers, 30 provider names, and 69 distinct cluster vectors. The selected quantizations were 32 FP8, six BF16, one FP4, one INT4, and 60 unknown. “Unknown” records absent endpoint metadata and does not identify a numerical format. The installed runtime pool preserves the generic persona path and the complete endpoint and cluster metadata for every row.

5 Execution and Records

The default full-catalog command runs from the repository’s `model-pool` directory:

```
make pool RUN_ID=pool-YYYYMMDDTHHMMSSZ
```

The Makefile builds the Go screen command and invokes every Python stage through `uv`. OpenRouter requests require an OpenRouter API key. Embedding requests require an OpenAI API key. The run identifier must name a new directory, which prevents one execution from mixing files with an earlier execution.

The default command inventories the full catalog, uses eight screening processes and eight evaluation processes, runs three evaluation trials, samples all 14 genes with five responses per configuration and gene, uses two concurrent gene processes, retains eight principal components, searches $k = 2$ through 20, and requests 100 pool entries with seed 0. The default score threshold is $S(c) \geq 0.90$; the recorded run supplied $S(c) > 0.70$. Command options expose the model roots, seeds, question set, prompt, trial count, filter, genes, persona, sample count, PCA dimension, cluster range, pool size, and timeouts.

Every stage has a bounded output set. Inventory keeps raw catalog responses and normalized endpoint rows. Screening keeps one directory per configuration with the direct response, Pi transcript, MCP log, result, and cost data. Evaluation keeps raw response rows, deterministic scores, exact request specifications, per-configuration logs, and a summary CSV. Later stages keep accepted and rejected configurations, completions and embeddings, PCA fits, cluster fits, aggregate decisions, equivalence groups, pool rows, and per-selection diagnostics.

The runner stops on a failed stage command, missing summary, incomplete catalog fetch, invalid record count, missing sample, missing embedding, impossible PCA dimension, invalid cluster data, or pool request larger than the eligible frame. Per-request completion errors during the gene stage remain in the records so other configurations and genes can finish. The cross-gene filter then excludes the affected configuration before PCA. This distinction treats one provider request as a candidate failure and a failed command or malformed stage output as a run failure.

6 Limitations

Provider catalogs and endpoint behavior change. A pass establishes the behavior of one configuration under one catalog snapshot, request shape, prompt wrapper, and set of trials. Later provider changes, model revisions, routing changes, and service failures require another inventory and evaluation.

The direct and Pi/MCP screens test the submission path with small required calls. They establish that the configuration completed those calls through the deployed protocol. They do not establish reliable use of every case tool, sustained tool use over a long matter, accurate legal analysis, or resistance to prompt-specific failures.

The deliberation score depends on 20 fixed questions and configured acceptable answers. Its threshold filters baseline behavior but does not estimate verdict accuracy over a case population. The scorer omits metadata-error rows from the deliberation denominator and records provider errors separately, so interpreting $S(c)$ without the operational filter would overstate configurations with missing responses.

The 14 genes define the observed behavior coordinates. Different prompts, personas, sampling parameters, embedding models, or sample counts can produce different clusters. Text embeddings measure semantic relatedness rather than legal independence, and PCA followed by K-means imposes Euclidean geometry and approximately center-based groups. Silhouette selection chooses a partition that separates these recorded embeddings; it does not establish that each cluster corresponds to a stable or legally relevant disposition.

Endpoint equivalence groups provider routes that share model identity, quantization, and modalities. Provider implementations can still produce different latency, availability, tool behavior, and response content. Selecting one operationally ranked representative reduces duplicate model configurations, while retaining the equivalent endpoints in the record, but it can remove provider-specific behavior from the sampling frame.

Uniform selection over observed cluster vectors gives rare vectors more weight than their frequency in the eligible catalog. A rare vector can reflect substantive difference, stochastic variation, or an embedding artifact. The one-per-model rule prevents duplicate model identifiers but permits several models from one vendor or model family, so it does not establish statistical independence among council members.

The scripts declare NumPy and scikit-learn without version pins. The recorded seed fixes pseudorandom choices, but exact PCA and K-means results can vary across dependency versions and numerical libraries.

The recorded costs cover evaluation responses and gene completions for which the providers returned cost observations. They exclude the unreported embedding total, screen executions across reruns, three gene completions without cost data, local computation, and storage. The reported dollar values therefore describe recorded API charges for specified stages rather than the complete cost of reproducing the pool.

7 Conclusion

The pipeline turns a provider catalog into an auditable adjudication pool by preserving endpoint identity, testing the runtime’s direct and agent-mediated tool paths, filtering repeated adjudication responses, and sampling across observed behavior-cluster vectors. The recorded run reduced 419 screened configurations to 265 complete configurations and selected 100 model identifiers across 30 providers and 69 cluster vectors. Its output supports exact request reconstruction and later analysis of every score, exclusion, cluster assignment, equivalence decision, and sampled row.

References

- [1] AgentCourt, “adj: model-pool implementation,” revision 36316f37ba577fdef69b8c6fe1247ce3509eab63, 2026. <https://github.com/agentcourt/adj/tree/36316f37ba577fdef69b8c6fe1247ce3509eab63/model-pool>
- [2] OpenRouter, “List all models and their properties,” API documentation, accessed August 30, 2026. <https://openrouter.ai/docs/api/api-reference/models/get-models>
- [3] OpenRouter, “List all endpoints for a model,” API documentation, accessed August 30, 2026. <https://openrouter.ai/docs/api/api-reference/endpoints/list-endpoints>
- [4] OpenRouter, “Provider routing,” developer documentation, accessed August 30, 2026. <https://openrouter.ai/docs/guides/routing/provider-selection>
- [5] OpenRouter, “Tool and function calling,” developer documentation, accessed August 30, 2026. <https://openrouter.ai/docs/guides/features/tool-calling>
- [6] Model Context Protocol, “Tools,” protocol specification, June 18, 2025. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>
- [7] M. Zechner and contributors, “Pi coding agent,” 2026. <https://github.com/badlogic/pi-mono/tree/main/packages/coding-agent>
- [8] OpenAI, “Vector embeddings,” API documentation, accessed August 30, 2026. <https://developers.openai.com/api/docs/guides/embeddings>
- [9] scikit-learn developers, “PCA,” documentation, accessed August 30, 2026. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
- [10] scikit-learn developers, “KMeans,” documentation, accessed August 30, 2026. <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>
- [11] scikit-learn developers, “silhouette_score,” documentation, accessed August 30, 2026. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html